

Spotify AI Support Agent — Report

REPORT — Spotify AI Support Agent (Hiver SDE Intern Take-Home)

1. Problem framing

The assignment: build a working AI support agent for a brand's Twitter customer-support dataset, with real infra (retrieval, reranking, agent orchestration, classification + reply drafting), and deliver it as if hiring for an AI-ramp SDE role.

Brand: **Spotify** / **@SpotifyCares**. Datasets: TWCS (Twitter Customer Support) and Banking77 (banking intent taxonomy, used for secondary intent calibration).

What "good" means for this brand. Spotify's social care answers thousands of public @-mentions daily; a wrong public reply is brand-visible, and a missed security/billing escalation is a trust incident. So "good" is asymmetric and I optimized accordingly:

- **Escalating something that needed a human is cheap; auto-handling one that didn't is expensive.** Hence precision-first routing with a deterministic safety gate, and 1.00 precision / 0.79 recall as the shipped tradeoff.
- **Every public reply must be grounded in something Spotify actually said.** Hence retrieval-grounded drafting with a hard "no invented URLs/amounts/ policy" rule, and groundedness as a first-class judge dimension (4.84/5).
- **The reply must sound like the brand, not like a chatbot.** Voice is learned from the historical archive itself, and scored (4.88/5).
- **First-response usefulness on routine asks** (the bulk of traffic): an actionable next step beats a full resolution — hence helpfulness (3.97/5) judged on "correct, actionable next step", not on solving.

What I chose not to build (deliberately):

- **Multi-turn conversation handling** — the golden set evaluates single messages by design; thread-context modelling is next-week item #1, not a half-built feature.
- **Real account tooling** — no account lookups, refunds, or plan changes; the agent asks for identifiers via DM the way @spotifycares actually does.
- **A web UI / live Twitter ingestion** — the deliverable is the pipeline and the proof it works, not a deployment shell.
- **Generative intent taxonomy discovery** — 9 intents derived from the data up front (see decision log) rather than letting the model invent labels per message; routable intents are the product.
- **Fine-tuning an embedding or LLM** — the assignment's leverage is in retrieval quality + routing policy, not in training; both embedding model (arctic-embed2, local) and LLM (Nemotron 3.5 lightning) are used as released.

Agent flow per incoming customer tweet:

1. **Classify** intent (9-way), sentiment, account identifiers, abuse, legal pre-flags. (LLM, JSON mode, Nemotron 3.5 lightning.)
2. **Retrieve** up to 5 most-similar resolved Spotify support cases via Weaviate Cloud (hybrid BM25+vector, MMR diversity, Jina reranking).
3. **Draft** a reply grounded in the historical resolutions (LLM, same constraints: no invented URLs / amounts / policy).
4. **Route** auto-handle vs escalate: the draft LLM proposes, then a deterministic escalation gate (`_esc_gate` in `src/agent.py`) overrides the draft whenever safety-critical signals fire — classifier abuse/pre-flag, account-compromise phrases, legal keywords (word-boundary matched), or billing-dispute patterns (refund, unauthorized charge, "full amount", charged-instead-of patterns).

2. Results (headline summary)

Full numbers: `eval/results/metrics.json` · summary: `eval/results/eval_summary.md`.

Metric	Agent	Simple baseline	Trivial baseline
Escalation accuracy	0.981	0.802	0.908
Escalation precision	1.000	0.077	0.0
Escalation recall	0.789	0.105	0.0
Escalation F1	0.882	0.089	0.0
Intent accuracy (9-way)	0.739	0.000	0.188
Reply presence (non-escalated)	1.000	0.884	1.000

LLM-as-judge (Nemotron 3-super-120b, rubric in `scripts/judge.py`):

Score (1-5)	mean	n
Helpfulness	3.97	192
Voice / brand tone	4.88	192
Groundedness / faithfulness	4.84	192
Escalation decision correctness	4.98	207
Overall	4.70	207 (97.1% ≥ 4)

Golden set: 207 hand-labelled out-of-sample examples — customer tweets @spotifycares that the brand **never answered**, so the retrieval index contains zero overlap (no cheating by copying archived replies). Built via stratified sampling (23 per intent bucket, 9 buckets, then hand-relabelled). Sampling/labelling methodology: `eval/GOLDEN_NOTES.md`.

Baseline context. "Trivial" = majority-intent, never escalate: it scores 90.8% escalation *accuracy* purely by class imbalance (188/207 are non-escalations) with zero recall — a reminder that accuracy alone is meaningless here. "Simple" = retrieval-only reply, keyword escalate: 7.7% precision (escalates on any profanity) and 0% intent accuracy (it labels everything with Banking77's bank-intent taxonomy — cross-corpus intent transfer fails completely, which is *why* the agent has its own 9-way taxonomy).

3. Iterating on failures (before → after)

The first full eval run surfaced two systematic bugs, both fixed and re-measured (pre-fix rows archived in `eval/results/precascade_*.jsonl`; flip derivation in `eval/results/regate_flips.json`):

Escalation	before	after
accuracy	0.894	0.981
precision	0.429	1.000
recall	0.474	0.789
F1	0.450	0.882
reply presence	0.942	1.000

- **Bug 1 — substring keyword matching.** `ESCALATION_KEYWORDS` included `"sue"`, and the gate matched substrings — so every message containing `"issue"` escalated. 5 false escalations. Fix: word-boundary matching (`(\b...\b)`).
- **Bug 2 — billing disputes never escalated.** All 6 gold-escalate billing disputes (student plan overcharged full price, charges never signed up for, "I'd like a refund", payment taken but account still free) were auto-handled with a cheerful "can you DM us your email?". Fix: a billing-dispute regex block in the gate. All 6 now escalate (precision untouched).

- **Bug 3 — abuse classifier over-triggering.** Frustrated profanity ("what the actual fuck why isn't album available") was flagged `abusive` and force-escalated; gold labels treat frustration $\neq$ abuse. Fix: a much stricter definition with contrastive examples in the prompt. 7 of 8 false flags dropped, while the one genuinely hostile message (a wall of 🙄) is still correctly caught.

4. Top-5 failure modes (from the final run, with real examples)

#	Failure class	Example (gold message)	Agent's output	Root-cause hypothesis
1	Ambiguous ownership compromise signals missed	"I keep seeing albums and songs I've never listened to... I think someone is accessing my account" (1814532)	auto-handled with a troubleshooting reply	Compromise phrased as suspicion ("I think"), no high-confidence keyword ("hacked"). Gate needs a graded suspicion tier.
2	Cross-system intrusion ambiguity	"I share wifi with other people and they are able to change music and stop mine" (2065702)	auto-handled as <code>device_integration</code>	Phrased as a device issue; only "someone is using my account" phrasing triggers the gate.
3	Multi-message fragments	"still. even after ive done all this" (350035)	generic "have you tried restarting?" — the prior context isn't in the golden set (single-message eval)	Context-less fragment needs conversation history; the pipeline treats it standalone (by design of this eval).
4	Feature-request → near-verbatim historical copy	"I heard we could have full screen album art with a button next to the play bar" (2052778)	asks for a screenshot about a <i>display</i> bug (copied context of a similar past case)	Retrieval matched an album-art complaint; the draft copied its <i>context</i> instead of the resolution pattern. Needs a "feature request $\neq$ bug" guard in the draft prompt.
5	Generic escalation acknowledgements	"hi, I have Spotify charges on my credit card. Although, I never signed up for it" (1809146)	correctly escalated — but historically the agent answered with an off-target regional-support redirect	Pre-gate draft quality on billing disputes was weak because almost no similar resolved cases exist in the archive; escalation is the right call, but first-response quality here is template-level.

5. Judge-human agreement (29-row hand-judged subset)

I hand-scored a 29-example subset (~14%) on the same rubric (`eval/human_judged.jsonl`) to check the LLM judge:

- **Escalation decision correctness:** judge vs human exact match **93.1%**, mean abs diff **0.14** (both 1-5). (Kappa is degenerate here — both raters sit in the ≥ 4 band on ~97% of rows; prevalence skew, documented in `metrics.json`.)
- **Helpfulness:** exact match 25%, mean abs diff **0.79** (1-5), binarized (≥ 4) $\kappa = 0.50$ — moderate agreement; the judge is ~0.5-1 point more generous than I am on "actionability".
- The judge **endorses the agent's routing on 96.6%** of subset rows where the agent's decision matches gold, and the agent's routing diverged from gold on **0 of 29** subset rows in the final run.
- Note the earlier version of this analysis mis-read the judge's `escalation_decision` (a correctness score) as a direction vote, yielding a nonsense "17% agreement". The fix is a small but real lesson: agreement metrics encode assumptions, and those assumptions need checking against the rubric.

6. What is misleading about my headline number?

The golden set uses **unanswered** inbound tweets — the brand never responded. That has two opposite effects that both distort the headline:

- It makes the set a **stress test on the hard tail**: unanswered tweets skew toward ambiguous, venting, or multi-message-fragment cases. 73.9% intent accuracy here is *not* the live-traffic rate — routine account-access/billing messages (where archived resolutions exist and the agent is strongest) are under-represented relative to the real stream.
- It makes **reply presence trivially achievable** (1.0) and makes the escalation base rate (19/207 $\approx$ 9%) lower than production, where compromise/abuse waves are common. Precision-1.0 on 15 escalations is a thin sample: ± 1 flip swings precision to 0.94.

In short: read this as a **hard-subset safety eval**, not a forecast of production accuracy. The recall misses (4/19) are all ambiguous-ownership compromises and context-less fragments — precisely the cases a real deployment would want flagged, which is why next-week list item #1 is conversation history.

7. Next week (if given one more)

1. **Conversation history**: real Twitter threads (multi-turn). The current agent processes single messages; the real @spotifycares inbox is threaded. Prior-turn compression + memory would fix failure mode #3 outright and #1/#2 partially.
2. **Graded suspicion tier in the gate**: "I think someone is using my account" deserves escalation even without a hard keyword — a medium-confidence tier with a "verify ownership first" macro reply, rather than the current binary keyword gate.
3. **Tool-calling LLM** for actual account lookups: hook the agent to a (mocked) account-data tool so billing disputes get a personalized first response instead of a template DM-request.
4. **Online evaluation**: shadow mode against the live support stream; measure handoff rate, response time, and CSAT-implied sentiment vs the human baseline.
5. **Active learning loop**: log deployed replies + agent confidence; flag low-confidence cases for human review and fold them into the golden set weekly.

8. Decision log

1. **One collection, two kinds** — the Weaviate Cloud free tier allows only one collection. Banking77 was inserted into `SpotifyCase` as `kind="banking77"`; the search layer filters on `kind` so the datasets never collide.
2. **Local ingest, cloud queries** — Ollama `snowflake-arctic-embed2` runs locally for ingest (no per-record API cost); the collection declares `text2vec-weaviate` with Snowflake `arctic-embed-l-v2.0` so Weaviate Cloud generates query vectors. Same model family = same vector space.
3. **Nemotron 3.5 lightning for agent, super-120b for judge** — the agent runs per-message and needs speed/cost control; the judge gets the heavier model for nuanced rubric scoring.
4. **`enable_thinking=false` + `json_mode=True`** — Nemotron 3.5 is a reasoning model; its thinking trace competes with `max_tokens` and truncates JSON mid-object. Disabling thinking and using structured output cut latency from minutes to seconds.
5. **OpenAILike $\rightarrow$ NvidiaNIM subclass** — the standard adapter rejects NIM model ids for function calling; subclassing with `is_function_calling_model=True` + `context_window=131072` makes LlamaIndex tool-calling work.
6. **LlamaIndex single-function-tool design** — three separate tools (classify/retrieve/draft) let the agent loop unboundedly; collapsing the deterministic pipeline into one `handle_support_case` tool capped iteration and matched the actual fixed pipeline.
7. **Golden set from never-answered inbound tweets** — using archived answers as test inputs would leak retrieval (the agent could copy them); unanswered tweets guarantee zero overlap.
8. **9-way intent, not 77-way** — Banking77's 77 categories are too fine for Twitter-length messages and don't map to support routing; the 9-way taxonomy comes from the Spotify data itself. (The simple baseline proves

the point: Banking77-as-is scores 0% on this set.)

9. **MMR balance 0.65** — pure max-relevance retrieval clusters around the same few templates; 0.65 pushes toward phrasing diversity the draft stage benefits from.
10. **Serial eval (--workers 1) + global request throttle** — NIM's free tier rate-limits aggressively under concurrency (429 stampedes); a 1.5s global minimum spacing between requests is faster in practice than concurrent failures + retries.
11. **Deterministic escalation gate on top of the LLM decision** — the draft LLM is good but not reliable on safety-critical routing; the gate (abuse/pre-flag, compromise phrases, legal keywords, billing disputes) overrides the draft deterministically. Every escalation carries its `escalation_rule` for auditability.
12. **Word-boundary keyword matching** — v1 substring matching made "sue" fire on "issue" (5 false escalations). A one-line fix with a 13-point F1 gain; kept as a reminder that the simplest bugs hide in the deterministic layers, not the model.

Appendix: reproducibility

```
# setup
pip install --user --break-system-packages -r requirements.txt
# env: export NVIDIA_API_KEY=... in ~/.bashrc; .env with WEAVIATE_URL, WEAVIATE_API_KEY, JINA_API_KEY
# ollama pull snowflake-arctic-embed2:latest

# index (if not already uploaded)
PYTHONPATH=. python3 scripts/ingest.py           # ~15-20 min (11.7k local embeds)
PYTHONPATH=. python3 scripts/ingest_banking77.py  # ~5 min

# agent smoke test
PYTHONPATH=. python3 scripts/agent_test.py        # ~20s

# full eval (resumable, ~2h with workers=1)
PYTHONPATH=. python3 scripts/run_eval.py --workers 1

# quick slice (<15 min): stratified 15-example fresh run, no cache
PYTHONPATH=. python3 scripts/run_eval.py --workers 1 --limit 15

# re-emit summary without re-running
PYTHONPATH=. python3 scripts/run_eval.py --only-report
```